Instead, using figure 3.6 as a reference, what we need to do is taking the subtree rooted in the new node's parent, and then rotating it so that the new node becomes its root (because it's certainly going to be the node with the highest priority).

Listing 3.5 describes the pseudocode for the insertion method, while figures 3.8 and 3.9 illustrates an example of inserting a new entry, to add "Beer" to the inventory with 20 units in stock.

First, we need to find the right place to insert the new entry in our existing inventory: this is done with a traversal of the tree, exactly like what happens with search, only keeping track of the parent of current node, in order to be able to add the new leaf.

Notice that we implemented this traversal using an explicit loop here, instead of recursion, to show to our readers how this approach works.

As we can see in the top half of figure 3.8, the first step is traversing the tree to search the right spot where we can add the new node as a leaf: we go left when we traverse "Flour" and "Butter", then right at "Bacon" (lines #4-#7 of listing 3.5).

Notice that for brevity we used a contracted naming notation in the figure: the newly added node, corresponding to variable `newNode` in listing 3.5, is denoted as `X` in the figures, and its parent with `P`.

At this point, when we exit the while loop, the temporary variable `parent` points to the node with key "Bacon", therefore the conditions at lines #8 and #10 are false, and we add the new node as a right child of `parent`, as shown in the bottom half of figure 3.8.

Looking at the example, we can also notice how the new node has a higher priority (a lower number of units in stock) than its